Mini Project 3 — Streamlit Deployment Documentation

Oguz Sinanoglu
EEP 596 — Agentic AI in FinTech
March 2026

Overview

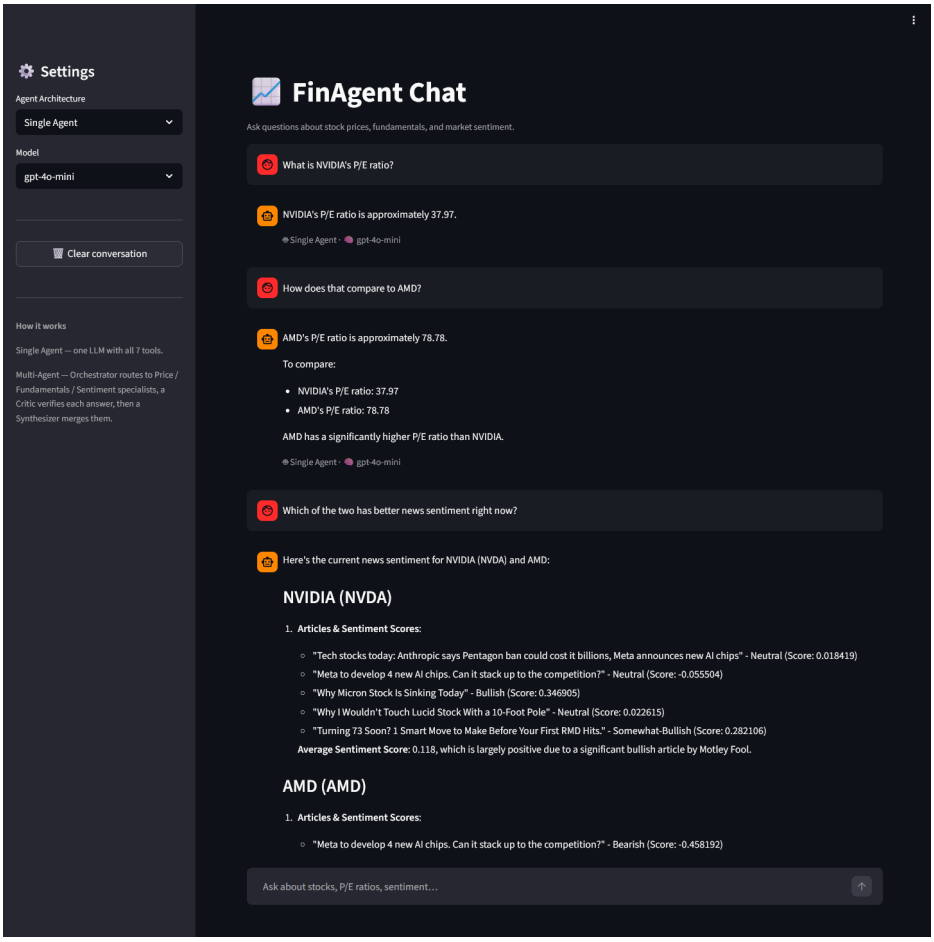
`app.py` is a Streamlit chat interface that wraps the Single Agent and Multi-Agent architectures built in `mp3_assignment.ipynb`. Both architectures share the same seven tool functions (AlphaVantage + yfinance + local SQLite) and the same `run_specialist_agent` agentic loop. The app adds sidebar controls and **conversational memory** — the last three exchanges are prepended to every new question so agents can resolve cross-turn references (“that”, “the two”, “it”) without re-stating context.

---

1. Sidebar — Agent & Model Selectors

The sidebar exposes two controls: an **Agent Architecture** dropdown (Single Agent / Multi-Agent) and a **Model** dropdown (gpt-4o-mini / gpt-4o). A **Clear conversation** button resets `st.session_state.messages` to `[]`.

Screenshot 1 — Single Agent · gpt-4o-mini (sidebar visible)



Sidebar and first exchange — Single Agent, gpt-4o-mini

Figure 1: Streamlit app sidebar showing agent selector set to "Single Agent", model selector set to "gpt-4o-mini", and the Clear conversation button. Chat area shows the beginning of a 3-turn follow-up conversation.

---

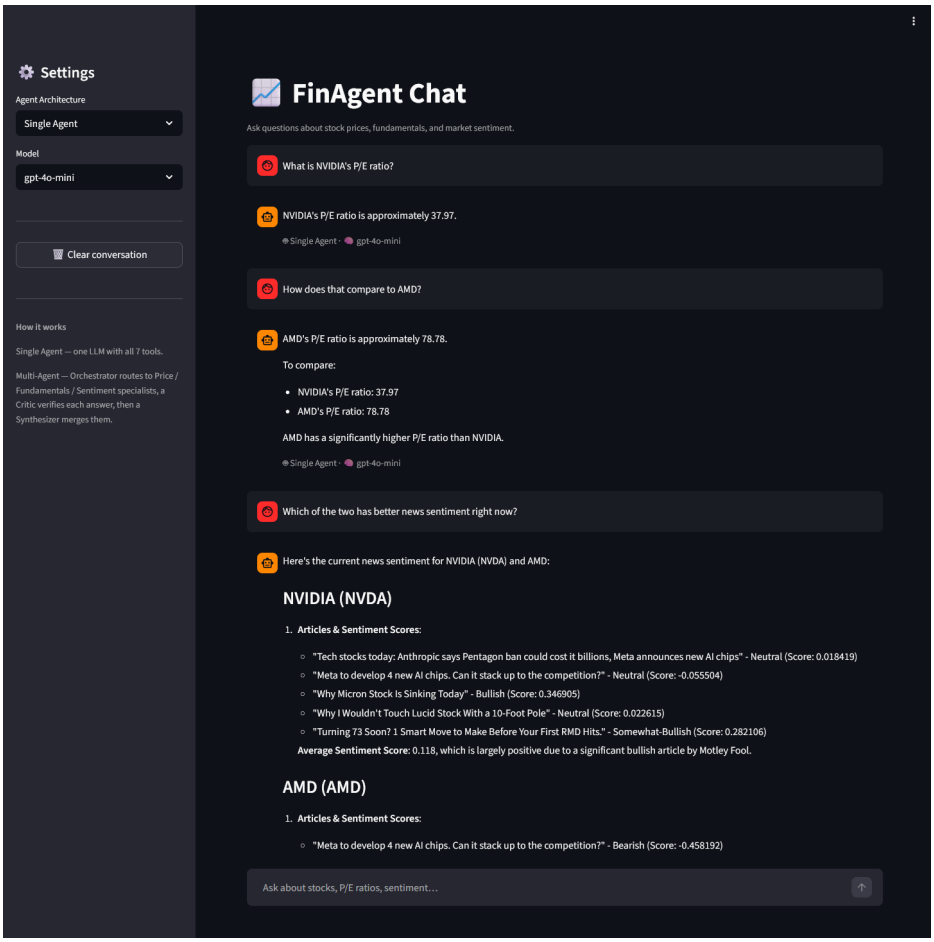
2. 3-Turn Follow-Up Conversation — Single Agent · gpt-4o-mini

The three exchanges below demonstrate that the agent correctly resolves pronoun references across turns. The conversation history (up to the last 6 messages) is prepended to each new question inside `_build_contextual_question()` in `finagents.py`.

Turn 1: Direct question establishing context — NVIDIA's P/E ratio.
Turn 2: Follow-up using "that" — comparison with AMD without

restating the subject.
Turn 3: Cross-domain follow-up using “the two” — sentiment for both tickers without repeating their names.

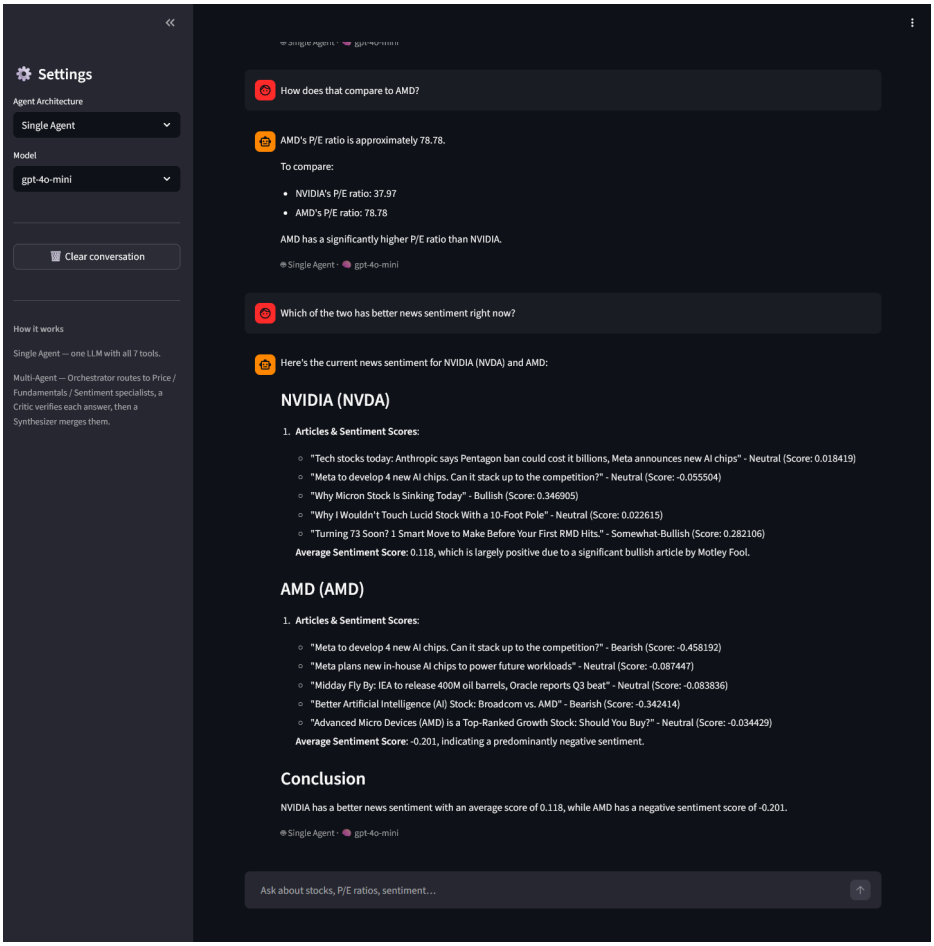
Screenshot 2 — Turns 1 & 2 — Single Agent · gpt-4o-mini



3-turn conversation turns 1 and 2 — Single Agent, gpt-4o-mini

Figure 2: Agent correctly answers “What is NVIDIA’s P/E ratio?” (Turn 1) and resolves “How does that compare to AMD?” (Turn 2) without the user restating which tickers to compare. The agent architecture and model are shown in the caption below each response.

Screenshot 3 — Turn 3 — Single Agent · gpt-4o-mini (continued)



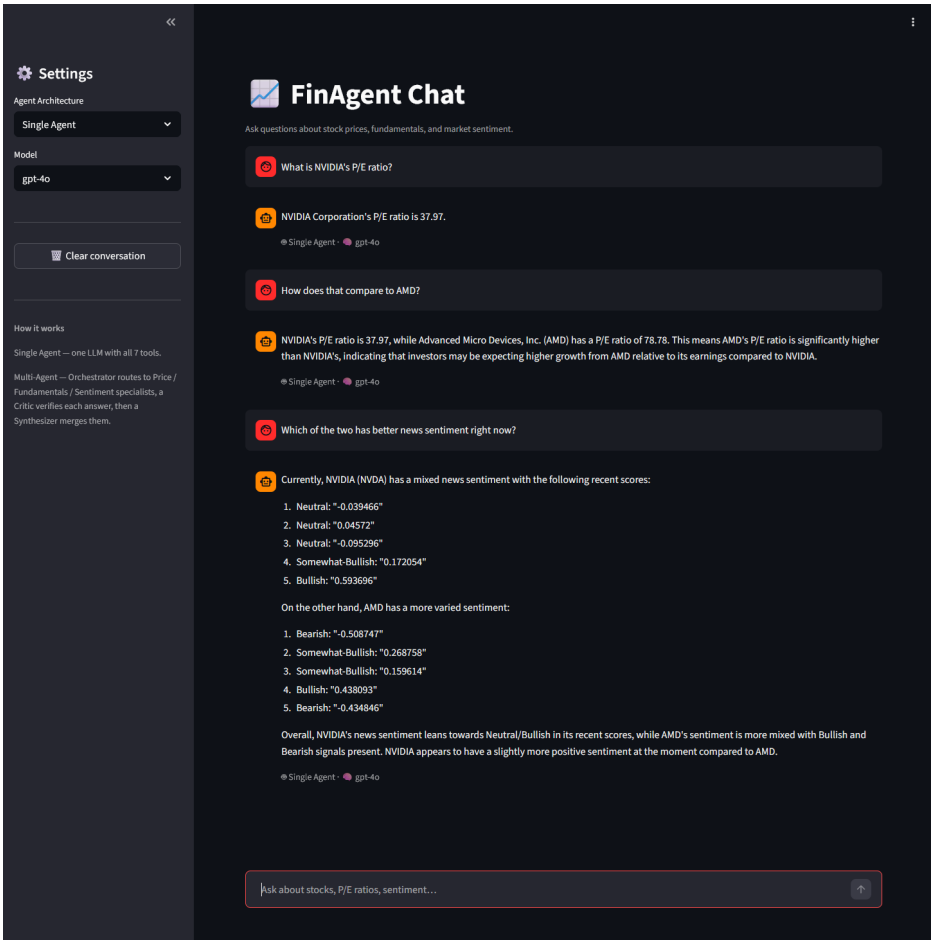
3-turn conversation turn 3 — Single Agent, gpt-4o-mini

Figure 3: Agent correctly resolves "Which of the two has better news sentiment right now?" (Turn 3) — it identifies "the two" as NVIDIA and AMD from prior context and fetches live sentiment for both tickers.

---

3. Single Agent — gpt-4o Example

Screenshot 4 — Single Agent · gpt-4o



Single Agent with gpt-4o model

Figure 4: Same Single Agent architecture running with the gpt-4o model selected in the sidebar. The agent architecture label ("Single Agent · gpt-4o") appears in the caption beneath the response.

---

4. Multi-Agent Architecture

Design: Orchestrator → Parallel Specialists → Critic → Synthesizer

```
User Question
      │
 Orchestrator ── analyzes question, selects specialists, detects
ranking dependency
      │
      ├─ Single-phase: ThreadPoolExecutor([selected agents])
```

```
────────────┐
     └─ Two-phase:  Phase 1 agent → extract tickers → Phase 2
parallel ┤

▼

Critic

│

Synthesizer

│

Answer
```

The system has five stages:

Stage 1 — Orchestrator (1 LLM call, no tools)

Receives the user question and decides which specialists to activate. Returns structured JSON specifying which agents to run and a tailored sub-task string for each:

```
        {
          "agents": ["Price", "Fundamentals"],
          "phased": false,
          "phase1_agent": "",
          "task_per_agent": {
            "Price":        "Fetch 1-year price performance for AAPL,
MSFT, GOOGL and rank by % change.",
            "Fundamentals": "Get P/E ratio for AAPL, MSFT, GOOGL from
Alpha Vantage."
          }
        }
```

Simple single-domain questions activate only one specialist; cross-domain questions activate multiple. The orchestrator also detects **ranking dependencies**: if a question first ranks by price and then asks for fundamentals or sentiment of the top results, it sets `"phased": true` so Phase 1 (Price) runs alone, extracts the ranked ticker list, and then Phase 2 agents receive only those tickers.

Stage 2 — Specialists (3 agents, fixed tool subsets)

Each specialist runs the shared `run_specialist_agent()` loop with its own system prompt and tool subset. Narrow tool subsets prevent wrong-tool choices — the main failure mode observed in the single agent on cross-domain questions.

Agent	Tools
Price Agent	`get_tickers_by_sector`, `get_price_performance`, `get_market_status`, `get_top_gainers_losers`
Fundamentals Agent	`get_company_overview`, `query_local_db`, `get_tickers_by_sector`
Sentiment Agent	`get_news_sentiment`, `query_local_db`

Stage 3 — Execution Mode

Single-phase (default): All selected specialists run concurrently via `ThreadPoolExecutor`. This keeps wall-clock time close to the single-agent baseline even when 2–3 specialists are active.

Two-phase (ranking dependency): Phase 1 (Price Agent) runs alone, its `raw_data` is parsed to extract the ranked ticker list, then Phase 2 specialists run in parallel targeting only those tickers. Required for questions like *"top 3 semiconductor stocks by 1-year return — get their P/E and sentiment"*.

Stage 4 — Critic (1 LLM call per specialist, no tools)

For each specialist, the Critic receives the agent's answer and its raw tool data. It checks internal consistency — do the numbers in the answer match the tool output? — and produces a `confidence` score (0.0–1.0) and `issues_found` list. This is hallucination detection without ground-truth access.

Stage 5 — Synthesizer (1 LLM call, no tools)

Merges all specialist answers into a single user-facing response. Applies **table format** for multi-ticker price comparisons; **prose format** for all other question types. Low-confidence results are noted explicitly rather than presented as fact.

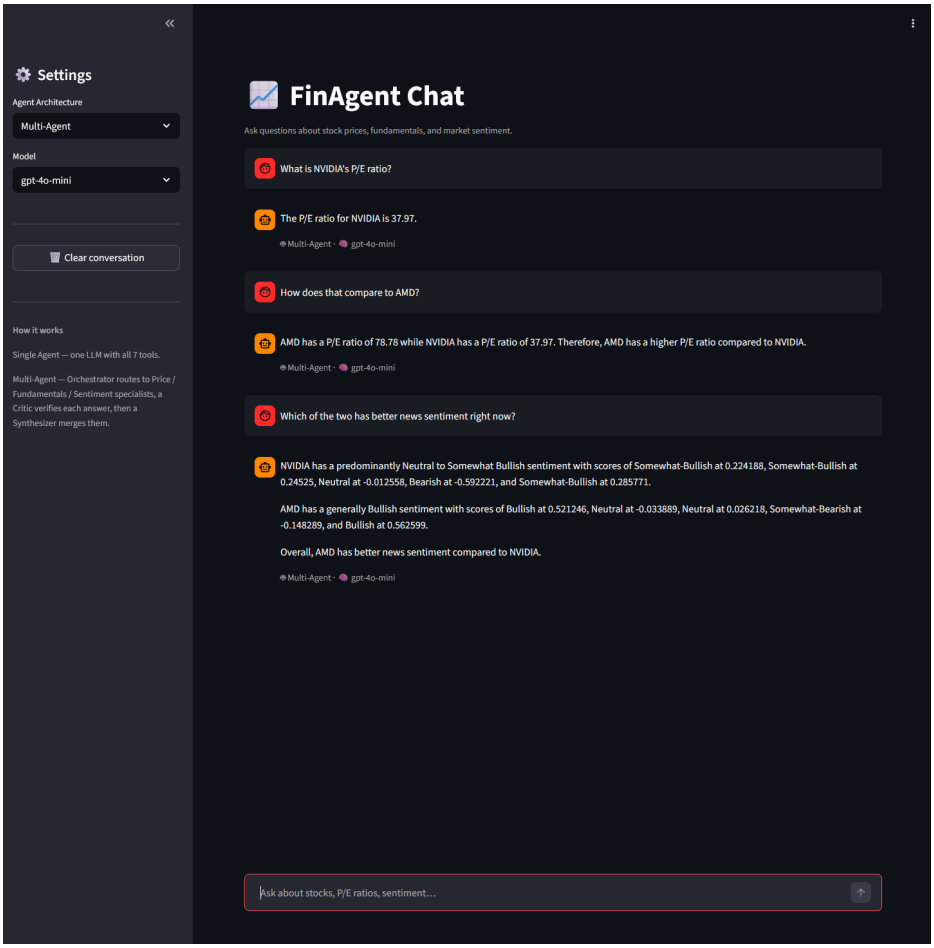
LLM Call Budget

Path	LLM calls
Single-phase, 1 specialist	Orchestrator + 1 specialist + 1 Critic + Synthesizer = **4**
Single-phase, 2–3 specialists	Orchestrator + 2–3 (parallel) + 2–3 Critics + Synthesizer = **6–8**
Two-phase, 3 specialists	

Path	LLM calls
	Orchestrator + Phase 1 + Phase 2 × 2 (parallel) + 3 Critics + Synthesizer = **8**

5. Multi-Agent — gpt-4o-mini Example

Screenshot 5 — Multi-Agent · gpt-4o-mini

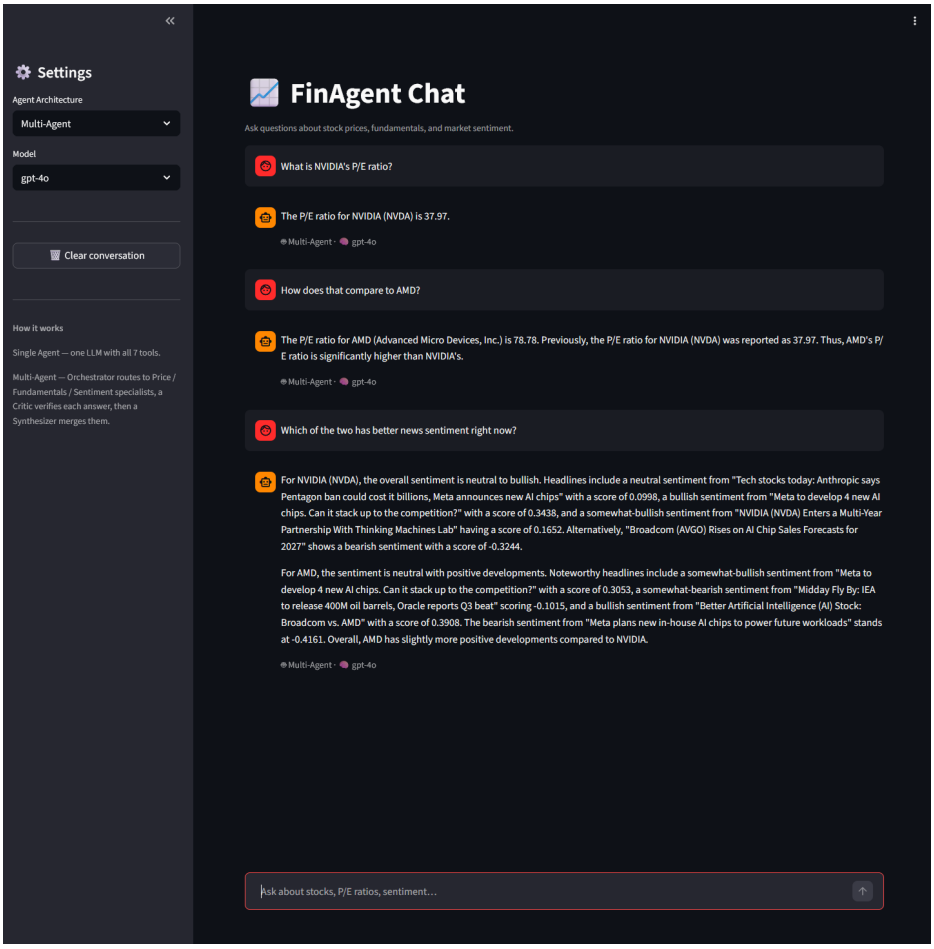


Multi-Agent with gpt-4o-mini model

Figure 5: Multi-Agent architecture with gpt-4o-mini. The Orchestrator routes this cross-domain question to two specialists (Price + Fundamentals) running in parallel. Each response is critic-verified before synthesis. The agent architecture label ("Multi-Agent · gpt-4o-mini") appears beneath the response.

6. Multi-Agent — gpt-4o Example

Screenshot 6 — Multi-Agent · gpt-4o



Multi-Agent with gpt-4o model

Figure 6: Multi-Agent architecture with gpt-4o selected in the sidebar. The more capable model produces higher-confidence specialist answers and cleaner synthesis, consistent with the benchmark evaluation showing +14.4 pp improvement on the multi-agent architecture when switching from gpt-4o-mini to gpt-4o.

---

7. Conversational Memory — Implementation Note

All conversation history is persisted in `st.session_state.messages` (Streamlit's standard session mechanism). On every new turn, `finagents._build_contextual_question()` prepends the last 6 messages (3 full exchanges) to the question text before calling the agent:

```
        "Conversation history (for context only):\n"
        "User: ...\nAssistant: ...\n"
        "...\n\n"
        "New question: <current question>"
```

This makes conversational context transparent to the LLM without any special memory management — the agent sees the history inline and can resolve any pronoun or reference from the last three turns.